Atelier 4 — Git et GitHub : travail collaboratif

Organisation : binôme — Développeur 1 et Développeur 2

Durée indicative : 1 h 30 à 2 h

Objectif : collaborer sur un même projet sans travailler directement sur la branche principale.

Contexte

Deux développeurs travaillent ensemble sur un petit site web. Le premier développeur initialise le projet et crée la page d'accueil. Il prépare ensuite une page de contact sur une branche dédiée. Le deuxième développeur rejoint le projet pour créer une page de promotion sur sa propre branche.

Les deux développeurs doivent publier leurs modifications sur GitHub, puis proposer leur intégration dans la branche principale à l'aide de Pull Requests.

Partie 1 — Création du dépôt distant et première version

Développeur 1

Étape 1. Créer un dépôt GitHub

1. Connectez-vous à GitHub.
2. Créez un nouveau dépôt nommé `atelier-collaboratif`.
3. Choisissez un dépôt public ou privé selon les consignes du formateur.
4. Ne cochez pas les options de création automatique d'un README, d'un `.gitignore` ou d'une licence : le dépôt doit être vide.
5. Récupérez l'URL SSH du dépôt.

Étape 2. Cloner le dépôt sur votre machine

Ouvrez un terminal et exécutez :

```
git clone git@github.com:VOTRE-UTILISATEUR/atelier-collaboratif.git
cd atelier-collaboratif
```

Remplacez `VOTRE-UTILISATEUR` par votre identifiant GitHub.

Vérifiez ensuite l'état du dépôt :

```
git status
git branch
```

```
git remote -v
```

Questions :

- Quel est le rôle de `git clone` ?

La commande `git clone` permet de copier un dépôt distant sur sa machine locale, en récupérant les fichiers du projet, son historique Git et les références des branches distantes.

- Quel dossier a été créé sur votre machine ?

Git crée un dossier nommé `atelier-collaboratif`, qui contient les fichiers du projet ainsi que le répertoire caché `.git`, contenant les données nécessaires au fonctionnement de Git.

- Que représente `origin` dans le résultat de `git remote -v` ?

`origin` est le nom par défaut donné au dépôt distant lors du clonage. Il permet d'identifier le dépôt GitHub associé au dépôt local.

- Quelle est la différence entre le dépôt local et le dépôt distant ?

Dépôt local	Dépôt distant
Situé sur la machine du développeur.	Hébergé sur un serveur, par exemple GitHub.
Permet de modifier les fichiers, créer des branches et effectuer des commits.	Permet de partager le projet avec les autres développeurs.
Les commits locaux ne sont pas automatiquement envoyés sur GitHub.	Reçoit les commits avec `git push`.
Les modifications distantes sont récupérées avec `git fetch` ou `git pull`.	Les modifications peuvent provenir de plusieurs collaborateurs.

Étape 3. Créer la page d'accueil

Dans le dossier du projet, créez un fichier `index.html` avec le contenu suivant :

```
<!DOCTYPE html>
<html lang="fr">
<head>
    <meta charset="UTF-8">
    <title>Atelier collaboratif</title>
</head>
<body>
    <h1>Bienvenue sur notre site</h1>
    <p>Projet réalisé en équipe avec Git et GitHub.</p>
</body>
</html>
```

Enregistrez le fichier, puis exécutez :

```
git status
git add index.html
git status
git commit -m "Ajouter la page d'accueil"
git push origin master
```

Si GitHub a créé la branche par défaut `main`, utilisez `main` à la place de `master` dans tout l'atelier.

Questions :

- Pourquoi exécuter `git add` avant `git commit` ?
- Que représente le commit créé ?
- Quel est le rôle de `git push` ?
- Que constatez-vous sur GitHub après le push ?

Partie 2 — Créer une branche pour la page Contact

Développeur 1

Étape 4. Créer une branche dédiée

Avant de commencer le développement de la page Contact, créez une nouvelle branche :

```
git checkout -b contact
```

Cette commande crée la branche `contact` et bascule immédiatement dessus.

Vérifiez la branche active :

```
git branch
```

L'astérisque `*` indique la branche courante.

Étape 5. Créer la page Contact

Créez un fichier `contact.html` :

```
<!DOCTYPE html>
<html lang="fr">
<head>
    <meta charset="UTF-8">
    <title>Contact</title>
</head>
<body>
    <h1>Contactez-nous</h1>
    <p>Email : contact@exemple.com</p>
</body>
</html>
```

Ajoutez et enregistrez la modification :

```
git add contact.html
git commit -m "Ajouter la page contact"
```

Étape 6. Publier la branche sur GitHub

Exécutez :

```
git push --set-upstream origin contact
```

Explication des éléments de la commande :

Élément	Signification
`git push`	Envoie les commits vers le dépôt distant
`origin`	Nom habituel du dépôt distant
`contact`	Branche locale à publier
`--set-upstream`	Configure la branche distante de suivi pour simplifier les prochains `git push` et `git pull`

À retenir : `origin` désigne le dépôt distant configuré localement, généralement le dépôt GitHub cloné. Ce n’est pas une commande ni une branche.

Après cette opération, vérifiez sur GitHub que la branche `contact` existe.

Partie 3 — Inviter le deuxième développeur

Développeur 1

Étape 7. Ajouter un collaborateur

Sur GitHub :

1. Ouvrez le dépôt `atelier-collaboratif`.
2. Accédez aux paramètres du dépôt (Settings).
3. Ouvrez la section de gestion des collaborateurs.
4. Invitez le compte GitHub du Développeur 2.
5. Le Développeur 2 accepte l'invitation.

Le Développeur 2 dispose maintenant des droits nécessaires pour collaborer sur le dépôt, selon les permissions accordées.

Partie 4 — Récupérer le projet et créer une branche Promotion

Développeur 2

Étape 8. Cloner le dépôt

Sur sa propre machine, le Développeur 2 exécute :

```
git clone git@github.com:UTILISATEUR-DEV1/atelier-collaboratif.git
cd atelier-collaboratif
```

Il vérifie ensuite :

```
git branch
git branch -r
git remote -v
```

Il constate que :

- sa branche locale courante est normalement `master` ou `main`, selon la branche par défaut du dépôt ;
- `origin/master` ou `origin/main` représente la branche distante principale ;
- `origin/contact` apparaît parmi les branches distantes si elle a bien été publiée ;
- la page `index.html` est présente dans la version principale clonée.

Attention : le clonage récupère les références des branches distantes, mais ne crée pas automatiquement une branche locale pour chacune d'elles.

Étape 9. Créer une branche Promotion

Le Développeur 2 doit partir de la branche principale, et non de `contact`, puisque son travail est indépendant.

```
git checkout master
git checkout -b promotion
```

Si la branche principale s'appelle `main`, remplacez `master` par `main`.

Étape 10. Créer la page Promotion

Créez `promotion.html` :

```
<!DOCTYPE html>
<html lang="fr">
<head>
    <meta charset="UTF-8">
    <title>Promotion</title>
</head>
<body>
    <h1>Nos promotions</h1>
    <p>Découvrez nos offres du moment.</p>
</body>
</html>
```

Ajoutez et validez le travail :

```
git add promotion.html
git commit -m "Ajouter la page promotion"
```

Publiez ensuite la branche :

```
git push --set-upstream origin promotion
```

La branche `promotion` est maintenant disponible sur GitHub.

Partie 5 — Créer les Pull Requests et intégrer les branches

L'objectif est de réunir les trois pages dans la branche principale, tout en passant par une revue du code.

Étape 11. Ouvrir la Pull Request Contact

Développeur 1

1. Ouvrez le dépôt sur GitHub.
2. Sélectionnez la branche `contact`.
3. Cliquez sur Compare & pull request.
4. Choisissez la branche de destination `master` (ou `main`).

5. Donnez un titre à la Pull Request : `Ajouter la page Contact`.
6. Créez la Pull Request.
7. Vérifiez les modifications, puis fusionnez-la (Merge pull request).

Étape 12. Ouvrir la Pull Request Promotion

Développeur 2

8. Ouvrez le dépôt sur GitHub.
9. Créez une Pull Request depuis `promotion` vers `master` (ou `main`).
10. Donnez le titre : `Ajouter la page Promotion`.
11. Examinez les fichiers modifiés.
12. Faites valider puis fusionnez la Pull Request.

Une Pull Request (PR) est une demande d'intégration de modifications. Elle permet de consulter le code et de le faire vérifier avant de fusionner une branche. Ce n'est pas une commande Git : elle se crée sur GitHub dans cet atelier.

Étape 13. Synchroniser les dépôts locaux

Après les fusions, chaque développeur peut récupérer la nouvelle version principale :

```
git checkout master
git pull origin master
```

Adaptez les commandes à `main` si nécessaire.

Le dépôt principal contient désormais les trois pages :

- `index.html`
- `contact.html`
- `promotion.html`

6. Schéma global du travail collaboratif



7 Questions de synthèse

1. Quelle est la différence entre un dépôt local et un dépôt distant ?
2. Pourquoi chaque développeur travaille-t-il sur une branche distincte ?
3. Quel est le rôle de `origin` dans `git push origin contact` ?
4. Pourquoi utilise-t-on `--set-upstream` lors du premier push d'une branche ?
5. Quelle est la différence entre `git push` et une Pull Request ?
6. Une Pull Request fusionne-t-elle automatiquement la branche ? Expliquez.
7. Pourquoi faut-il récupérer la branche principale après une fusion ?
8. Combien de branches ont été créées au cours de l'atelier, et quelles sont leurs fonctions ?